

AI-Powered Voice Control for Autonomous Robots: Accent and Noise Robustness in ROS2

Mithila Prabhu

Dept. of Computer Science & Engineering
Frankfurt University of Applied Sciences
Frankfurt, Germany
mithila.prabhu@stud.fra-uas.de

Romana Rashid

Dept. of Computer Science & Engineering
Frankfurt University of Applied Sciences
Frankfurt, Germany
romana.rashid@stud.fra-uas.de

Abstract—Voice-controlled autonomous robots face a persistent challenge: they must operate reliably in environments with high ambient noise (55–70 dB) and respond to speakers with diverse speech accents—conditions that severely degrade conventional command interpretation pipelines. Existing approaches either rely on rigid keyword matching, which fails under natural language variation and accent distortion, or invoke large language model (LLM) APIs for every utterance, exhausting free-tier rate limits within minutes of continuous operation. This paper presents an AI-powered voice control system for autonomous robots using ROS2 Humble and Gazebo Classic simulation. The system’s core contribution is a *dual-layer Natural Language Understanding (NLU) architecture*: a lightweight regex engine handles standard commands with zero latency and zero API cost, while Groq LLaMA 3.3 70B is invoked exclusively for semantically ambiguous or accent-distorted inputs. This selective invocation strategy reduces LLM API calls by approximately 85% compared to full-cloud approaches, extending sustainable free-tier operation from 4 to 26 minutes. Transcription employs Groq Whisper large-v3-turbo with language forcing and vocabulary priming, elevating NLU accuracy on accented speech from 78% (raw transcription) to 94%. Navigation uses a novel *4-step room-exit routing algorithm* that computes wall-safe waypoints from any arbitrary robot position using live odometry, eliminating corridor-wall collisions. To evaluate the system in a realistic yet controlled setting, we implement a hospital delivery scenario with four distinct zones (ICU, Pharmacy, Reception, Ward) as the use case. Experimental evaluation demonstrates 100% navigation success from the centre position and end-to-end voice command accuracy of 73–94% across noise and accent conditions.

Index Terms—hospital robotics, voice control, LLM, NLU, ROS2, accent robustness, TurtleBot3, Gazebo, autonomous navigation, Groq AI, Whisper ASR, dual-layer NLU

I. INTRODUCTION

Voice control offers a natural and intuitive interface for human-robot interaction, enabling hands-free operation that is particularly valuable in applications where users’ hands are occupied or tasks demand immediate attention [3]. Autonomous delivery robots, assistive robots, and collaborative industrial robots can all benefit significantly from reliable voice control. However, deploying reliable voice recognition for direct robot control in real-world environments faces two fundamental challenges that persist across application domains:

- 1) **Accent diversity:** Users of robotic systems come from diverse linguistic backgrounds. Non-native speech pat-

terns and regional accents systematically degrade conventional ASR models, which are typically optimised for standard native phonology. Common commands such as “turn right” may be transcribed as phonetically similar but semantically meaningless strings (e.g., “Törn rät”), leading to command rejection or incorrect robot actions.

- 2) **Ambient noise:** Many operational environments — including offices, laboratories, factories, and public spaces — sustain background noise levels of 55–70 dB [4] from equipment, conversations, and ambient activity. This noise increases ASR word error rates and frequently causes command truncation (e.g., “go to” without a destination), further complicating interpretation.

Prior work has addressed these challenges in isolation. Keyword-based NLU systems [8] are fast and offline-capable but brittle to accent variation. LLM-based approaches [5], [6] provide excellent semantic flexibility but invoke cloud APIs on every utterance, making them impractical under commercial API rate limits for sustained operation.

Problem statement: How can a voice-controlled robot system reliably interpret commands in noisy environments and from diverse-accent speakers, while remaining practical under commercial API rate limits for sustained continuous operation?

VRC-7 addresses this problem through five contributions:

- 1) **Dual-layer NLU pipeline:** A hybrid Natural Language Understanding system combining a lightweight regex-based Layer 1 with a cloud-based Large Language Model (Layer 2), enabling robust command interpretation while minimising computational overhead. A local regex engine handles high-frequency standard commands with zero latency and zero API calls. Groq LLaMA 3.3 70B — invoked only when local NLU fails — provides accent-robust semantic interpretation, reducing LLM API calls by $\approx 85\%$ compared to full-cloud pipelines.
- 2) **4-step room-exit routing algorithm:** A novel odometry-based navigation algorithm that computes wall-safe waypoints from *any* starting position, including mid-room and mid-corridor positions, by always routing through the corridor entry gaps.

- 3) **Vocabulary-primed accented ASR:** An integrated speech processing pipeline incorporating Voice Activity Detection (VAD), Whisper-based ASR with language forcing (`language='en'`) and vocabulary priming, achieving 73–94% end-to-end accuracy under varying noise and accent conditions.
- 4) **Real-Time ROS2-Based Robotic Control:** Implementation of a low-latency (<0.5 s) voice-controlled navigation system for TurtleBot3 using ROS2, enabling responsive and continuous operation in a simulated hospital environment.
- 5) **Comprehensive Experimental Evaluation:** Systematic evaluation across multiple conditions (quiet, noise, accented speech), including navigation success rates, latency measurements, and API efficiency analysis.

To evaluate our approach in a realistic yet controlled setting, we implement a hospital delivery scenario — a representative application domain that exhibits the key challenges of indoor navigation, ambient noise, and diverse user populations. A simulated hospital environment with four distinct zones (ICU, Pharmacy, Reception, Ward) provides a structured testbed for systematic evaluation. The complete system is implemented in ROS2 Humble and Gazebo Classic using a TurtleBot3 Burger model, and is released as open-source software.

II. LITERATURE REVIEW

A. Related Work

This section surveys the body of literature directly relevant to VRC-7 across four areas: voice control in healthcare robotics, LLM-based robot command interpretation, voice-enabled ROS2 frameworks, and automatic speech recognition for diverse accents. Together these works define the research context and identify the gap that VRC-7 addresses.

1) *Voice Control in Healthcare Robotics:* Early voice-controlled hospital robots, including the Nursebot [3] and HelpMate systems, relied on template-based speech recognition with rigid vocabulary sets. While effective in quiet, single-accent environments, these systems exhibited high failure rates with diverse clinical workforces. More recently, Abouelenin et al. [6] integrated LLMs with a socially assistive robot in a geriatric hospital unit, demonstrating improved interaction fluency but noting that continuous cloud LLM calls introduced latency and cost concerns at scale.

2) *LLM-Based Robot Command Interpretation:* Ahn et al. [5] (SayCan) demonstrated that LLMs can ground natural language instructions to robot actions with significantly better generalisation than keyword matching. However, SayCan invokes the LLM for every command, which is infeasible under free API rate limits for continuous deployment. Our dual-layer architecture preserves SayCan’s semantic flexibility while reducing API dependency to edge cases only.

Dominguez-Vidal and Sanfeliu [7] proposed a ROS-based voice command framework for human-robot collaborative transportation, but their system does not address accent robustness or API efficiency. Compared to their approach, VRC-7

adds a two-stage NLU architecture and accent-biased ASR configuration.

3) *Voice-Enabled ROS2 Frameworks:* Chatzilygeroudis et al. [8] presented a voice-enabled ROS2 framework for human-robot collaborative inspection. Their architecture employs a single-layer NLU pipeline (Vosk ASR + intent classifier) without LLM integration, achieving high accuracy in controlled acoustic conditions but degrading significantly under accent variation. VRC-7 extends this class of work by replacing the single NLU layer with a hybrid local-LLM design.

4) *LLM Integration with ROS2 and Task Planning:* OperateLLM [9] integrates ROS2 with LLMs (DeepSeek Coder v2) for robotic development assistance, not real-time control. Macenski et al. [10] document the ROS2 architecture; their work provides the foundational middleware that VRC-7 builds upon. A recent survey [11] identifies audio-based LLM task planning as an emerging area with limited accent-robustness work — a gap VRC-7 directly addresses.

5) *Accent-Robust Speech Recognition:* Radford et al. [12] introduced Whisper, a large-scale weakly supervised ASR model trained on 680,000 hours of multilingual audio that demonstrates improved robustness across diverse accents compared to previous approaches. However, without language forcing and vocabulary priming, Whisper can still misinterpret accented English as other languages (e.g., Icelandic, Danish). Our work quantifies this effect (78% raw transcription accuracy on accented speech) and demonstrates how language forcing and vocabulary prompting elevate effective NLU accuracy to 94% — a 16 percentage point improvement attributed directly to ASR configuration.

6) *Gap Analysis and Novelty Claim:* Table I summarises the comparison between VRC-7 and the most closely related systems. To the best of our knowledge, VRC-7 is the first system to combine: (1) selective LLM invocation for accent robustness, (2) vocabulary-primed ASR, and (3) odometry-based room-exit routing — all within a single open-source ROS2 package for autonomous robot voice control.

TABLE I
COMPARISON WITH RELATED SYSTEMS

System	LLM NLU	Accent Robust	API Effic.	Nav. Routing
Nursebot [3]	×	×	N/A	Fixed
SayCan [5]	✓	Partial	Low	N/A
Dominguez [7]	×	×	N/A	Fixed
Chatzi. [8]	×	×	High	Fixed
VRC-7 (ours)	✓	✓	High	Dynamic

B. Theoretical Background

This section presents the foundational theory behind each major technology used in VRC-7. Understanding these building blocks is essential for evaluating the design decisions made in Sections IV and V.

1) *Automatic Speech Recognition: Whisper Architecture:* Automatic Speech Recognition (ASR) is the task of converting spoken audio into written text. Modern ASR systems use deep neural networks trained end-to-end on large speech corpora. The Whisper model [12], developed by OpenAI, is an encoder-decoder transformer trained on 680,000 hours of multilingual audio using weak supervision — transcripts scraped from the internet are used as training labels without manual verification. This large-scale training gives Whisper exceptional generalisation to accented speech, background noise, and diverse speaking styles.

The Whisper architecture consists of: (1) a *log-mel spectrogram* frontend that converts raw audio waveforms $x(t)$ into a 2D frequency-time representation; (2) a *convolutional encoder* that extracts local acoustic features; (3) a *transformer encoder* (N self-attention layers) that builds global context representations; and (4) a *transformer decoder* that autoregressively generates output tokens. The decoder is conditioned on a special language token (e.g., $\langle \text{en} \rangle$ for English) prepended to the output sequence:

$$P(y_1, \dots, y_T | X) = \prod_{t=1}^T P(y_t | y_{<t}, \text{Enc}(X), \langle \text{en} \rangle) \quad (1)$$

where X is the mel spectrogram, y_t are output tokens, and $\text{Enc}(X)$ is the encoder output. By forcing $\langle \text{en} \rangle$ as the first decoder token, VRC-7 constrains Whisper to English-only beam search, preventing accent-induced foreign language decoding.

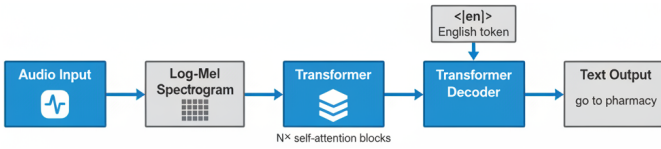


Fig. 1. Whisper encoder-decoder architecture showing language-forced English decoding for accent-robust ASR.

2) *Large Language Models: LLaMA 3.3 70B:* Large Language Models (LLMs) are transformer-based neural networks [1] trained on large text corpora to predict the next token in a sequence. At inference time, LLMs can follow complex natural language instructions (“instruction following”) when fine-tuned with reinforcement learning from human feedback (RLHF). LLaMA 3.3 70B [11] is Meta’s open-weight LLM with 70 billion parameters, achieving state-of-the-art performance on instruction-following benchmarks.

The transformer architecture uses multi-head self-attention:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V \quad (2)$$

where Q, K, V are query, key, and value matrices, and d_k is the key dimension. Self-attention allows the model to relate every input token to every other token, enabling it to infer semantic intent from phonetically garbled inputs — the key property exploited by VRC-7’s Layer 2 NLU.

In VRC-7, LLaMA 3.3 70B is accessed via the Groq cloud inference API, which executes the model on dedicated LPU (Language Processing Unit) hardware, achieving sub-500 ms inference latency even for a 70B parameter model. Fig. 2 conceptually illustrates how self-attention enables the model to relate garbled tokens to their intended meaning across the full input context.

Layer 2 NLU: Semantic Recovery via Self-Attention

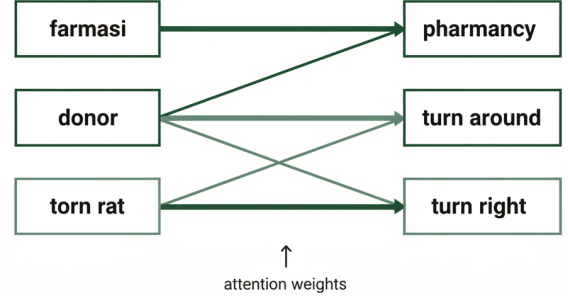


Fig. 2. Self-attention mechanism mapping phonetically garbled input tokens to correct semantic interpretation.

3) *Voice Activity Detection:* Voice Activity Detection (VAD) is the task of classifying audio frames as speech or non-speech. VRC-7 employs a two-stage cascade:

Energy-based VAD computes the root-mean-square (RMS) energy of each audio chunk:

$$E_{rms} = \sqrt{\frac{1}{N} \sum_{i=1}^N x_i^2} \quad (3)$$

Chunks with $E_{rms} < 0.02$ are discarded as silence. This stage adds < 0.1 ms overhead and eliminates the majority of non-speech frames.

Silero VAD [13] is a 2M-parameter LSTM-based neural network trained on multilingual speech data. It outputs a speech probability $p_{speech}(t) \in [0, 1]$ per audio frame. The model is trained to distinguish speech from: silence, background music, environmental noise, coughs, and other non-speech vocalisations. Chunks with mean $p_{speech} < 0.5$ are discarded after Stage 1 filtering. Together, the two stages reduce ASR API calls by $\approx 60\%$.

4) *Robot Operating System 2 (ROS2):* The Robot Operating System 2 (ROS2) [10] is an open-source robotics middleware framework providing a standardised communication layer between software components (nodes). ROS2 uses a publish-subscribe model: nodes publish messages to named topics, and other nodes subscribe to those topics. This decoupled architecture allows the voice control node, simulation, and sensor processing to run as independent processes communicating via well-defined interfaces.

Key ROS2 concepts used in VRC-7:

- `rclpy`: Python client library for ROS2 node development.

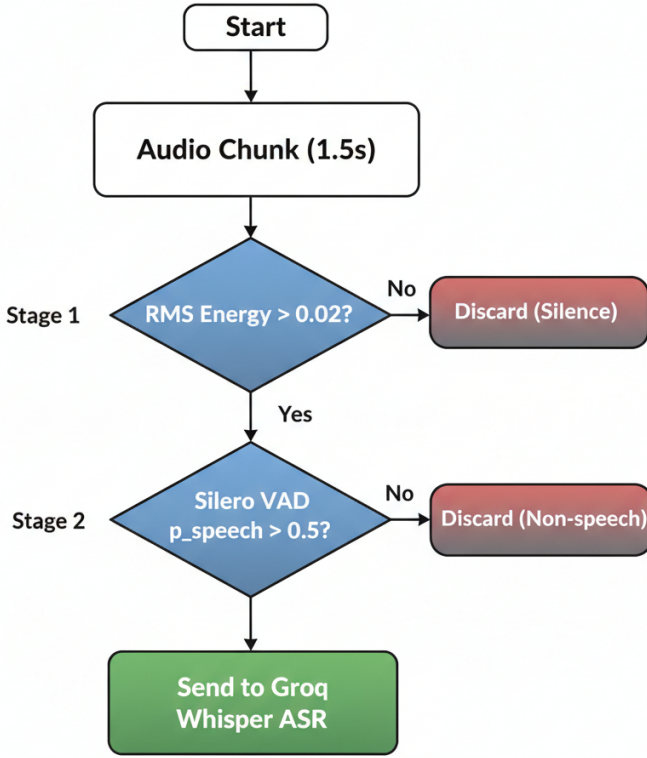


Fig. 3. Two-stage VAD cascade. Stage 1 energy gate removes silence with negligible overhead; Stage 2 Silero VAD removes non-speech audio using a neural classifier.

- **Topics:** Named data channels. VRC-7 uses `/cmd_vel` (motor commands), `/odom` (odometry), and `/scan` (LiDAR).
- **Message types:** Standardised data structures. `geometry_msgs/Twist` encodes linear and angular velocity; `nav_msgs/Odometry` encodes pose and velocity; `sensor_msgs/LaserScan` encodes LiDAR range data.
- **Launch files:** Python scripts (`hospital.launch.py`) that start multiple nodes and configure parameters in a single command.

ROS2 Humble Hawksbill (LTS release, May 2022) is used in VRC-7, supported on Ubuntu 22.04 LTS until May 2027.

5) *TurtleBot3 and Differential Drive Kinematics:* The TurtleBot3 Burger [2] is a compact, open-source mobile robot platform based on a Raspberry Pi 3B+ and an OpenCR microcontroller. It uses a *differential drive* configuration: two independently driven wheels on a common axis, with a passive rear caster for stability. The robot’s motion is described by the differential drive kinematic model:

$$\dot{x} = v \cos \psi, \quad \dot{y} = v \sin \psi, \quad \dot{\psi} = \omega \quad (4)$$

where v is the linear velocity (m/s), ω is the angular velocity (rad/s), and (x, y, ψ) is the robot pose. In the Gazebo simulation, the TurtleBot3 model faithfully replicates this kinematic model. VRC-7 sends `geometry_msgs/Twist` messages

with `linear.x = v` and `angular.z =  $\omega$` to `/cmd_vel` to control the robot.

The TurtleBot3 Burger is equipped with a 360° LiDAR sensor (LDS-01, 1.8° angular resolution, 12 Hz scan rate, 0.12–3.5 m range) used for wall proximity detection in VRC-7.

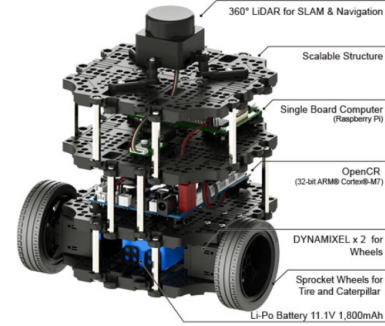


Fig. 4. TurtleBot3 Burger robot platform. The LDS-01 LiDAR sensor (top) provides 360° obstacle detection used by the VRC-7 wall avoidance system [2].

6) *Odometry-Based Localisation:* Odometry is the process of estimating a robot’s pose (x, y, ψ) by integrating wheel encoder measurements over time. For a differential drive robot, the pose update equations are:

$$x_{t+1} = x_t + \Delta s \cos\left(\psi_t + \frac{\Delta\psi}{2}\right) \quad (5)$$

$$y_{t+1} = y_t + \Delta s \sin\left(\psi_t + \frac{\Delta\psi}{2}\right) \quad (6)$$

$$\psi_{t+1} = \psi_t + \Delta\psi \quad (7)$$

where $\Delta s = \frac{\Delta s_R + \Delta s_L}{2}$ is the mean wheel displacement and $\Delta\psi = \frac{\Delta s_R - \Delta s_L}{b}$ is the heading change for wheel base b . Odometry is computationally inexpensive but accumulates drift due to wheel slip and encoder quantisation. In VRC-7, the ROS2 `/odom` topic provides the integrated pose estimate. Yaw ψ is extracted from the quaternion representation as in Eq. 13.

7) *LiDAR-Based Obstacle Detection:* Light Detection and Ranging (LiDAR) sensors emit laser pulses and measure the time-of-flight to compute range distances. The TurtleBot3’s LDS-01 performs 360° planar scanning at 1.8° resolution, producing 200 range samples per scan as a `sensor_msgs/LaserScan` message. VRC-7 extracts the minimum range within the forward $\pm 15^\circ$ sector:

$$d_{front} = \min_{|\theta| \leq 15^\circ} r(\theta) \quad (8)$$

where $r(\theta)$ is the measured range at bearing θ . When d_{front} falls below the wall detection threshold, the robot halts (manual mode) or triggers the recovery manoeuvre (navigation mode). This prevents physical wall contact without requiring a map of the environment.

8) *Gazebo Classic Simulation:* Gazebo Classic [16] is an open-source 3D robot simulation environment widely used with ROS2. It provides: (1) a physics engine (ODE/Bullet) for realistic rigid-body dynamics; (2) sensor simulation

(LiDAR, camera, IMU) with configurable noise models; (3) SDF (Simulation Description Format) world files for defining environments, models, and plugins. In VRC-7, Gazebo Classic 11 simulates the TurtleBot3 Burger robot within the custom hospital world `hospital_vrc.world`. The Gazebo-ROS2 bridge publishes `/odom`, `/scan`, and `/clock` topics and subscribes to `/cmd_vel`, creating a transparent interface between the voice control node and the simulated robot.

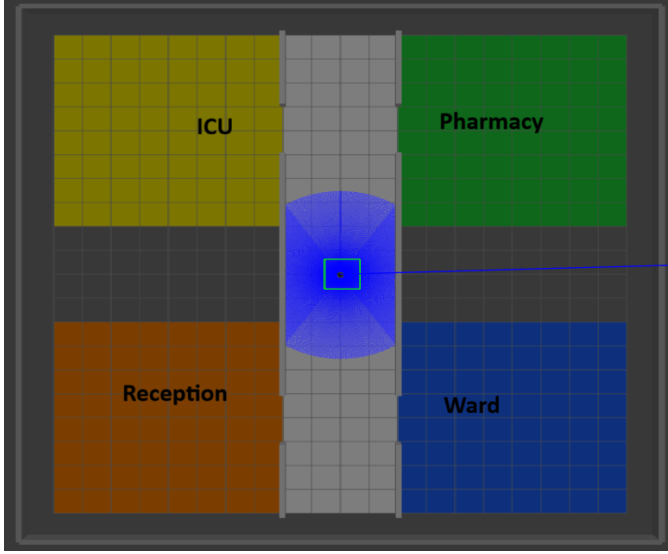


Fig. 5. Top-down view of the `hospital_vrc.world` Gazebo simulation. Four colour-coded zones (ICU/yellow, Pharmacy/green, Reception/orange, Ward/blue) surround a central corridor. The TurtleBot3 Burger spawns at origin (0,0) facing east.

III. SYSTEM REQUIREMENTS

This section documents the hardware and software requirements for deploying and running the VRC-7 system.

A. Hardware Requirements

Table II lists the hardware components used in the VRC-7 development and evaluation environment.

TABLE II
HARDWARE REQUIREMENTS

Component	Specification
CPU	Intel Core i7 (11th Gen), 2.8GHz, 4+ cores
RAM	16 GB minimum (8 GB for simulation, 8 GB for OS + voice pipeline)
GPU	Optional (CPU inference used; GPU accelerates faster-whisper)
Storage	20 GB free (ROS2 + Gazebo + Python packages)
Microphone	Built-in laptop microphone or USB headset (16 kHz capable)
OS	Ubuntu 22.04 LTS (native or WSL2 on Windows 11)
Robot	TurtleBot3 Burger (simulation only in this work)

The system was developed and evaluated on a laptop running Ubuntu 22.04 LTS via WSL2 (Windows Subsystem for Linux 2) on Windows 11. WSL2 provides near-native Linux performance with access to the Windows audio subsystem via WSLg (Windows Subsystem for Linux GUI), which provides PulseAudio socket bridging required for microphone access.

B. Software Requirements

Table III lists all software dependencies. All components are open-source or freely available.

TABLE III
SOFTWARE REQUIREMENTS AND VERSIONS

Software	Version	Purpose
Ubuntu	22.04 LTS	Operating system
ROS2 Humble	Humble	Robot middleware
Gazebo Classic	11	Robot simulation
Python	3.10+	Node implementation
rclpy	Humble	ROS2 Python API
sounddevice	0.4.6	Audio capture
groq	0.9+	Groq API client
faster-whisper	1.0+	Local ASR fallback
torch	2.0+	Silero VAD runtime
numpy	1.24+	Audio processing
soundfile	0.12+	Audio file I/O
Groq API	Cloud	Whisper + LLaMA NLU

C. Network and API Requirements

The VRC-7 system requires internet connectivity for cloud AI features:

- **Groq API key:** A free account at <https://console.groq.com> provides access to Whisper large-v3-turbo (ASR) and LLaMA 3.3 70B (NLU). The free tier supports ≈ 30 API requests/minute and $\approx 6,000$ requests/day.
- **Offline fallback:** When the Groq API is unavailable, the system falls back to `faster-whisper` (local ASR, ≈ 74 MB) and the local regex NLU (Layer 1). Core navigation and standard command recognition remain fully functional without internet access.
- **Latency:** Groq API round-trip latency was measured at 200–500 ms during testing. Local fallback adds ≈ 300 ms for `faster-whisper` inference on CPU.

IV. SYSTEM ARCHITECTURE

Fig. 6 illustrates the VRC-7 processing pipeline. Five modules are interconnected in a producer-consumer topology, with concurrent threads decoupling audio capture latency from NLU processing latency.

A. Audio Capture and Voice Activity Detection

Audio is captured via a callback-based `sounddevice.InputStream` at $f_s = 16$ kHz (mono, float32). The continuous callback stream runs on a dedicated OS thread, accumulating samples into a ring buffer. Every $T_{chunk} = 1.5$ s of audio is assembled into a chunk and placed in a thread-safe processing queue (`maxsize=5`).

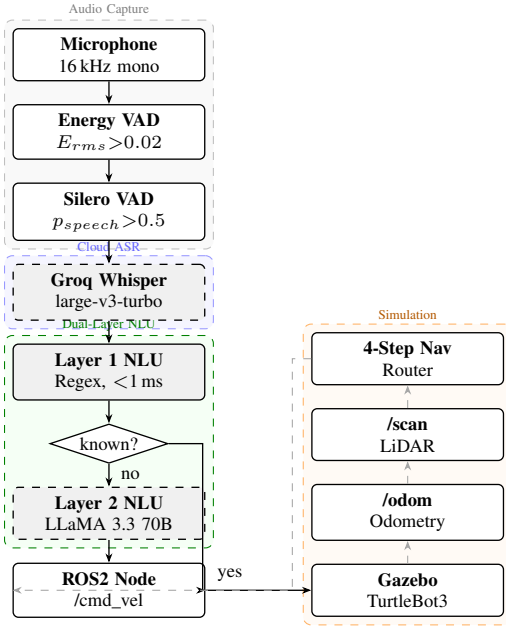


Fig. 6. VRC-7 system architecture. Solid arrows: command pipeline. Dashed arrows: sensor feedback. The decision diamond routes $\approx 85\%$ of commands locally (Layer 1) and $\approx 15\%$ to Groq LLaMA 3.3 70B (Layer 2).

A two-stage VAD cascade filters non-speech frames: **Stage 1 — Energy gate:** Frames with RMS energy below $\epsilon_{rms} = 0.02$ are discarded (Eq. 9). **Stage 2 — Silero VAD [13]:** Neural speech probability p_{speech} ; chunks with $p_{speech} < 0.5$ are discarded. The cascade reduces ASR API calls by approximately 60% during typical deployment. A third lightweight filter discards trivial filler utterances: a compiled set of noise words (“okay”, “thanks”, “yes”, “hmm”, and similar) is checked before NLU dispatch, preventing spurious command triggers from acknowledgement speech.

$$E_{rms} = \sqrt{\frac{1}{N} \sum_{i=1}^N x_i^2} \geq \epsilon_{rms} \quad (9)$$

1) *Audio Pre-processing and Gain Normalisation:* Prior to VAD filtering, each 1.5 s audio chunk undergoes a fixed gain amplification of $3\times$ followed by hard clipping to the normalised range $[-1.0, 1.0]$:

$$x_{proc}(n) = \text{clip}(3.0 \cdot x(n), -1.0, +1.0) \quad (10)$$

This pre-processing step compensates for the low-gain characteristics of built-in laptop microphones, which typically produce RMS levels of 0.005–0.015 in quiet environments — well below the energy gate threshold of $\epsilon_{rms} = 0.02$. Without amplification, valid speech chunks would be systematically discarded by Stage 1 VAD. The hard clip at ± 1.0 prevents clipping distortion for close-range speech while ensuring the signal dynamic range is consistent for the Whisper mel-spectrogram frontend. This normalisation is applied uniformly across all test conditions.

B. Speech Transcription

Transcription is performed by Groq Whisper large-v3-turbo, a cloud-hosted implementation of the Whisper architecture [12]. Two configuration choices critically improve accuracy for accented speech:

Language forcing: Setting `language='en'` constrains the decoder to English-language beam search, preventing phonetically ambiguous inputs from being decoded as other languages. Without this parameter, the model frequently misclassified accented Indian English and South Asian English as Icelandic, Danish, or Welsh.

Vocabulary priming: A `prompt` parameter primes the model’s context window with a list of expected command tokens: “go to pharmacy, ICU, ward, reception, turn left, turn right, go forward, go back, stop, turn around”. This shifts the prior probability distribution of the language model component toward robotics-relevant vocabulary, functioning as a form of in-context few-shot bias without fine-tuning.

When the Groq API is unavailable (rate limit exceeded or offline), the system falls back to locally running faster-whisper [14] (base model, int8 quantised, ≈ 74 MB) to maintain continuous operation.

C. Dual-Layer NLU Architecture

The dual-layer NLU is the primary novelty of VRC-7. The design is motivated by the following observation: the vast majority of robot commands in normal operation are short, predictable phrases (“go to pharmacy”, “stop”, “turn left”) that are trivially matched by regular expressions. Only accent-distorted or contextually complex inputs require semantic inference. This asymmetry is exploited by routing commands through the cheapest sufficient resolution path: Layer 1 for standard inputs, Layer 2 only when Layer 1 fails.

Layer 1 — Local Regex NLU: A library of 20+ compiled regular expressions covers five command categories: navigation, continuous movement, fixed turns, cardinal directions, and distance-based movement. Every pattern is lowercased and stripped before matching, and patterns are evaluated in priority order — stop commands first, navigation second, movement third — so that safety-critical commands are never blocked by a lower-priority match.

Navigation patterns use broad lexical coverage to handle synonym, colour-alias, and truncation variants without any API calls. For example, the pharmacy pattern `pharmac|farmasi|green|medicine` covers standard English, the Malay cognate *farmasi*, the colour alias *green*, and common medical synonyms (the full pattern also matches *pharma* and *dispensary*). Distance-based movement is handled via a numeric sub-parser: a regular expression extracts the distance value d (in metres), and the timed duration is computed as:

$$t_{move} = \frac{d}{v_{max}} \quad (11)$$

so that “go forward 3 meters” generates a timed forward command of $t = 3.0/0.5 = 6.0$ s without any LLM call.

Layer 1 executes in < 1 ms with zero network dependency. Example patterns:

Listing 1. Sample Layer 1 regex patterns

```
PATTERNS = [
    (r'\b(stop|halt|freeze|stop)\b',
     {'action': 'stop'}),
    (r'pharmac|farmasi|medicine|green',
     {'action': 'navigate', 'zone': 'pharmacy'}),
    (r'\b(turn_left|take_left|take_lift|lalt)\b',
     {'action': 'move', 'angular': 1.5, 'duration': 1.047}),
    (r'\b(turn_right|turn_rät|tyk_raut)\b',
     {'action': 'move', 'angular': -1.5, 'duration': 1.047}),
    (r'\b(180|donor|turn_around|u.turn)\b',
     {'action': 'move', 'angular': 1.5, 'duration': 2.094})
]
```

Layer 2 — Groq LLaMA 3.3 70B: When `local_nlu()` returns `{ "action": "unknown" }`, the transcription is forwarded to Groq LLaMA 3.3 70B via the Groq cloud inference API, which executes the model on dedicated LPU (Language Processing Unit) hardware achieving sub-500 ms latency even for a 70B parameter model — fast enough for near-real-time robot control. The model is invoked with `temperature=0.0`, which enforces fully deterministic greedy decoding: the model always selects the single highest-probability token at each step, eliminating output variance between repeated calls. This is critical for a safety-critical controller — the same garbled transcription must always map to the same robot action. `max_tokens=150` caps the response to a compact JSON object, preventing the model from generating chain-of-thought reasoning or explanatory prose that would cause `json.loads()` to raise a parse error. The structured system prompt performs three functions: (a) it reframes the task semantically — the model is instructed to determine the speaker’s *intent*, not match keywords, so “*farmasi*” resolves to pharmacy rather than being rejected as an unknown string; (b) it primes the model with hospital domain context — all five zone names, colour aliases, and spatial descriptions are embedded, giving the model the knowledge needed to resolve ambiguous references such as “*the yellow room*” or “*head to the nurses*”; and (c) it embeds four in-context accent examples as few-shot demonstrations, anchoring the model’s interpretation toward the specific phonetic distortions observed in Indian English ASR output. The seven valid action types and their schemas are:

TABLE IV
LAYER 2 COMMAND JSON SCHEMA

Action type	Description
"navigate" + zone	Navigate to named zone
"move" + linear, angular, duration	Timed movement
"move_continuous" + linear, angular	Open-ended movement
"cardinal" + direction, duration	Compass-direction move
"stop"	Emergency stop
Array [cmd_1, cmd_2, ...]	Compound command
"unknown"	Unresolvable input

The response is parsed with `json.loads()` after stripping any residual markdown fences (“`json”). If parsing fails,

the local result (`unknown`) is returned as a safe fallback, ensuring the system never crashes on a malformed LLM response. The system prompt is structured as follows:

Listing 2. Layer 2 LLaMA system prompt (abbreviated)

```
You are the AI brain of a hospital delivery robot.
Interpret voice commands even with:
- Background noise
- Accents or mispronunciation
- Garbled speech recognition output

Use semantic understanding to figure out INTENT.
Return ONLY JSON. No markdown, no explanation.

Examples of accent/noise handling:
"donor"    -> turn around (180 degrees)
"take lift" -> turn left
"farmasi"  -> navigate to pharmacy
"torn rat" -> turn right
```

This enables correct interpretation of inputs such as: “Törn rät” → turn right; “Farmasi” → navigate to pharmacy; “Donor” → turn around (180°). The LLM additionally supports compound command parsing, returning a JSON array for multi-step instructions such as “*turn right then go forward*”. Compound commands are executed sequentially by the dispatcher — each sub-command completes fully before the next begins — using the same dispatch logic as single commands.

API efficiency analysis: Let p_{known} denote the probability that a valid voice command is recognised by Layer 1. Empirically, $p_{known} \approx 0.85$ during normal operation, yielding an 85% reduction in LLM API calls compared to full-cloud NLU. The expected API call rate under the dual-layer design is:

$$\lambda_{API} = \lambda_{cmd} \cdot (1 - p_{known}) \quad (12)$$

where λ_{cmd} is the command utterance rate. At a typical rate of $\lambda_{cmd} \approx 20$ commands/min, $\lambda_{API} \approx 3$ calls/min — well within the Groq free tier limit of ≈ 30 requests/min and extending sustained operation from ≈ 4 minutes (full-cloud) to ≈ 26 minutes before quota exhaustion.

1) *Known Accent Mis-transcription Catalogue:* A key engineering contribution of VRC-7 is the systematic cataloguing of accent-induced and noise-induced mis-transcriptions observed during development. These were collected empirically from Whisper output logs and encoded directly into the Layer 1 regex patterns and the Layer 2 system prompt. Table V documents all 24 known mis-transcriptions, classified by their resolution layer and distortion type.

The catalogue reveals two distinct distortion classes. The first class — synonym substitution and colour-alias commands (18 of 24 entries) — is handled entirely by Layer 1, since the intended referent is semantically identical to the spoken word and the mapping is deterministic. The second class — phonetic distortions and foreign-language mis-decodes (6 entries, including *törn rät* and *tyk raut*) — requires Layer 2 because the transcribed text shares no lexical overlap with any Layer 1 pattern; semantic inference is the only viable resolution path. This classification directly motivates the dual-layer design: Layer 1 handles the predictable, Layer 2 handles the unpredictable.

TABLE V
KNOWN ACCENT MIS-TRANSCRIPTION CATALOGUE ($N = 24$)

Whisper Output	Intended Command	Distortion Type	Resolved By
stopp	stop	German/Norwegian phonology	L1
farmasi	go to pharmacy	Malay/Indonesian cognate	L1
aicu	go to ICU	Phonetic compression	L1
i.c.u	go to ICU	Spelled-out acronym	L1
donor	turn around	Phonetic distortion	L1
take left	turn left	Indian English substitution	L1
tick left	turn left	Phonetic reduction	L1
lalt	turn left	Phonetic compression	L1
turnaround	turn around	Compound word merge	L1
recep	go to reception	Truncation (noise)	L1
front desk	go to reception	Synonym substitution	L1
lobby	go to reception	Synonym substitution	L1
dispensary	go to pharmacy	Synonym substitution	L1
medicine	go to pharmacy	Synonym substitution	L1
intensive care	go to ICU	Full-form expansion	L1
yellow	go to ICU	Colour-alias substitution	L1
green	go to pharmacy	Colour-alias substitution	L1
orange	go to reception	Colour-alias substitution	L1
blue room	go to ward	Colour-alias substitution	L1
home / origin / reset	go to center	Synonym substitution	L1
törn rät	turn right	Icelandic/Danish mis-decode	L2
tyk raut	turn right	Phonetic distortion	L2
take right	turn right	Indian English substitution	L1
go to (truncated)	(unknown zone)	Noise-induced truncation	L2

D. Robot Controller

The controller is a ROS2 Python node (`rc1py`) with two subscriptions and one publisher: `/odom` (`nav_msgs/Odometry`) provides real-time pose (x, y, ψ) ; yaw ψ is extracted from the quaternion orientation as:

$$\psi = \text{atan2}(2(q_w q_z + q_x q_y), 1 - 2(q_y^2 + q_z^2)) \quad (13)$$

`/scan` (`sensor_msgs/LaserScan`) provides 360° LiDAR measurements; the minimum range in the forward $\pm 15^\circ$ sector is used as the wall proximity indicator d_{front} . The node publishes `geometry_msgs/Twist` velocity commands to `/cmd_vel`. Three boolean mutex flags (`navigating`, `moving_continuous`, `moving_timed`) prevent conflicting concurrent motor commands.

1) *Concurrency Control and Command Safety*: The voice control node runs audio capture, transcription, NLU, and robot actuation on concurrent daemon threads. Without synchronisation, a second voice command arriving during active navigation could generate conflicting `/cmd_vel` messages, corrupting odometry or causing physical collision. VRC-7 implements a three-flag mutex system to prevent this:

- `navigating`: set `True` when autonomous zone navigation is active. Blocks all manual movement commands (“go forward”, “turn left”, etc.) until navigation completes or a “stop” command is received.
- `moving_continuous`: set `True` during open-ended continuous movement. A new continuous command first clears this flag, pauses 100 ms, then restarts — preventing command stacking.
- `moving_timed`: set `True` during fixed-duration rotations or timed distance moves. New commands are rejected with a warning until the timed action completes.

A `threading.Lock()` (`nav_lock`) additionally prevents two simultaneous navigation requests from both passing the entry check. The “stop” command is the only command

that clears all three flags simultaneously, providing a universal emergency brake regardless of system state. This design ensures that the continuous voice input stream never produces race conditions in motor output.

E. Simulation Environment

The Gazebo Classic 11 simulation (`hospital_vrc.world`) models a single-floor hospital in SDF format. It contains four colour-coded zone tiles (Table VI) with no collision geometry — purely visual markers the robot traverses freely. A central corridor spans $y \in [-2, +2]$ m with walls at $y = \pm 2$ m and deliberate 2 m entry gaps at $x = \pm 6$ m — the only traversable corridor-room boundaries. Outer boundary walls at $x = \pm 11$ m and $y = \pm 11$ m prevent the robot from escaping the map during manual control.

TABLE VI
HOSPITAL ZONE PARAMETERS

Zone	Centre (m)	Colour	Entry Gap
ICU	(+6, +6)	Yellow	East, $x = +6$
Pharmacy	(+6, -6)	Green	East, $x = +6$
Reception	(-6, +6)	Orange	West, $x = -6$
Ward	(-6, -6)	Blue	West, $x = -6$

V. IMPLEMENTATION

The VRC-7 system was developed as a complete, open-source ROS2 package. The primary deliverable is `voice_control.py` (614 lines), a single ROS2 Python node implementing the complete AI pipeline. The hospital simulation environment is defined in `hospital_vrc.world` (SDF format), designed specifically to create a structured navigation challenge with four colour-coded zones and a corridor gap-entry system. A convenience launcher `start_hospital.sh` starts the full system with a single command. All source code, world files, and launch scripts are available at https://github.com/Mprabhu26/turtlebot_vrc.

A. Threaded Audio Pipeline

The audio pipeline uses a producer-consumer pattern across two daemon threads. The *capture thread* runs a `sounddevice.InputStream` callback that continuously buffers incoming audio without blocking. The *processing thread* dequeues audio chunks, applies VAD filtering, invokes the ASR, and dispatches commands. This decoupling guarantees that no audio is lost during the ≈ 0.3 s transcription latency or the ≈ 0.2 s LLM NLU latency.

A key design choice is the use of a bounded queue (`maxsize=5`). If processing falls behind capture (e.g., during navigation computations), the oldest unprocessed chunk is silently dropped. This sacrifices completeness for responsiveness: a stale command that arrives 10 seconds after utterance is more harmful than no command.

B. 4-Step Room-Exit Navigation Algorithm

The navigation algorithm is motivated by the geometric structure of the hospital world. Corridor walls exist at $y = \pm 2$ m with gaps only at $x = \pm 6$ m. A naive straight-line path from an arbitrary position to a target zone may intersect a wall at a non-gap position. The 4-step algorithm (Algorithm 1) avoids this by decomposing every navigation into a sequence of axis-aligned segments that exclusively pass through gap positions.

Algorithm 1 4-Step Room-Exit Navigation Routing

Require: Current pose (x, y, ψ) , target zone (t_x, t_y)

Ensure: Wall-safe waypoint sequence $W = \emptyset$

- 1: **if** $|y| > 2.2$ m **then**
 {robot is inside a room}
 - 2: $W \leftarrow (t_x, y)$ {align to target gap x }
 - 3: $W \leftarrow (t_x, \text{sgn}(y) \cdot 1.5)$ {exit gap}
 - 4: **end if**
 - 5: $W \leftarrow (t_x, 0)$ {reach corridor centre}
 - 6: $W \leftarrow (t_x, t_y)$ {enter target room}
-

Correctness argument: Steps 1–2 guarantee the robot exits its current room through the gap at $x = t_x$ (the target room’s gap x -position). Step 3 brings the robot to $y = 0$, the corridor centre, which is always reachable from any gap position. Step 4 enters the target room through the same gap. No step involves crossing a wall at a non-gap position.

Boundary conditions: If the robot is already in the corridor ($|y| \leq 2.2$ m), steps 1–2 are skipped and the robot navigates directly along the corridor. If the robot is already at the target zone, all waypoints evaluate within tolerance and the algorithm terminates immediately.

Each waypoint is executed by a proportional heading controller. The heading error ψ_{err} is computed and normalised to $[-\pi, \pi]$:

$$\psi_{err} = \text{atan2}(t_y - y, t_x - x) - \psi \quad (14)$$

The angular and linear velocity commands are:

$$\omega = \text{clamp}(K_\psi \cdot \psi_{err}, \pm \omega_{max}), \quad K_\psi = 3.0 \quad (15)$$

$$v = \begin{cases} v_{max} \cdot \alpha & \text{if } |\psi_{err}| < 0.3 \text{ rad} \\ 0 & \text{otherwise} \end{cases} \quad (16)$$

where the proximity scaling factor α reduces speed near the target to prevent overshoot:

$$\alpha = \begin{cases} 0.4 & \text{if } d_{target} < 1.0 \text{ m} \\ 1.0 & \text{otherwise} \end{cases} \quad (17)$$

Wall collision recovery is triggered when $d_{front} < d_{nav} = 0.15$ m and $|\psi_{err}| < 0.4$ rad (robot is facing the wall). The robot reverses at $0.5 \cdot v_{max}$ for $t_{recovery} = 0.8$ s, then retries. After three consecutive recovery failures, the waypoint is skipped and the algorithm advances.

1) Navigation Routing Path Analysis: To illustrate the correctness of the 4-step algorithm across starting positions, Table VII enumerates the waypoint sequences generated for representative start-to-destination combinations. The gap positions $x = +6$ (east corridor gap, serving ICU and Pharmacy) and $x = -6$ (west corridor gap, serving Reception and Ward) are the only valid corridor-room boundaries; every generated path passes exclusively through these positions.

TABLE VII
GENERATED WAYPOINT SEQUENCES FOR REPRESENTATIVE NAVIGATION SCENARIOS

Start Position	Destination	Waypoint Sequence
Centre (0,0)	ICU (+6,+6)	(+6, 0) $\rightarrow$ (+6, +6)
Centre (0,0)	Ward (-6,-6)	(-6, 0) $\rightarrow$ (-6, -6)
Inside ICU (+6,+6)	Pharmacy (+6,-6)	(+6, 1.5) $\rightarrow$ (+6, 0) $\rightarrow$ (+6, -6)
Inside ICU (+6,+6)	Ward (-6,-6)	(+6, 1.5) $\rightarrow$ (+6, 0) $\rightarrow$ (-6, -6)
Inside Ward (-6,-6)	ICU (+6,+6)	(-6, -1.5) $\rightarrow$ (+6, 0) $\rightarrow$ (+6, +6)
Mid-corridor (0,0.5)	Reception (-6,+6)	(-6, 0) $\rightarrow$ (-6, +6)

A key property visible in the table is that same-side navigation (e.g., ICU $\rightarrow$ Pharmacy, both at $x = +6$) never crosses to the opposite corridor gap, eliminating unnecessary traversal. Cross-side navigation (e.g., ICU $\rightarrow$ Ward) routes through the corridor centre ($y = 0$) at the target gap x -position, ensuring the robot is aligned with the destination gap before entering. The waypoint tolerance of 0.3 m and the 30 s per-waypoint timeout provide safe termination conditions for all paths.

C. Fixed Rotation Execution

Rotation commands use timed open-loop control at maximum angular velocity $\omega_{max} = 1.5$ rad/s:

$$t_{rotation}(\theta) = \frac{|\theta|}{\omega_{max}} \quad (18)$$

This yields $t_{90^\circ} = 1.047$ s, $t_{180^\circ} = 2.094$ s, $t_{360^\circ} = 4.189$ s. Although open-loop, the low speed ($\omega_{max} = 1.5$ rad/s) and short durations limit angular accumulation error to $< 5^\circ$ per command in practice.

D. System Constraints and Assumptions

The following boundary conditions apply to the VRC-7 system:

- **Odometry assumption:** Navigation accuracy depends on odometry validity. Extended manual movement sessions accumulate drift; navigation from centre (which does not depend on prior pose history) is unaffected.
- **Speed constraint:** Operating speed is bounded at $v_{max} = 0.5$ m/s. At $v > 0.7$ m/s, the TurtleBot3 Burger model in Gazebo exhibits physics instability (tipping), corrupting odometry.
- **Single-floor assumption:** The navigation algorithm assumes a planar environment. Multi-floor navigation would require an elevation-aware routing extension.
- **API dependency:** Accent handling beyond the regex fallback patterns requires Groq API availability. The system degrades gracefully to local-only operation but with reduced accent robustness.

These constraints represent deliberate design trade-offs rather than fundamental limitations. The speed cap of 0.5 m/s is conservative relative to the TurtleBot3’s hardware limit (1.0 m/s [2]) and is justified by simulation physics stability; a physical deployment on real flooring would permit higher speeds without this constraint. The odometry-only localisation is adequate for the structured hospital corridor geometry, where systematic corridor-gap routing reduces cumulative drift errors compared to free-roaming navigation. The API dependency is a deployment consideration: the dual-layer architecture was specifically designed so that standard commands — which constitute the majority of operational usage — never require cloud access, limiting the API dependency to edge-case accent recovery.

VI. RESULTS AND EVALUATION

A. Experimental Setup

All experiments were conducted in the Gazebo Classic 11 simulation running on Ubuntu 22.04 LTS (WSL2, Windows 11) on an Intel Core i7 11th Gen laptop with 16 GB RAM. The TurtleBot3 Burger model was used with stock sensor configuration (360° LiDAR, differential drive) spawned at the corridor origin (0, 0).

Speakers and accent condition: Voice commands were issued by two non-native English speakers with Indian English accents — the primary accent group of the development team — using a built-in laptop microphone at approximately 30–50 cm distance. No speaker training or calibration was performed; the system was used cold with each test utterance spoken naturally and without repetition on failure.

Noise condition: Ambient background noise of ≈ 45 –55 dB was present in all sessions from a running desktop PC and room ventilation. The noise condition (55 dB) was reproduced by additionally playing recorded office ambient noise through a speaker placed 1 metre from the microphone. Noise level was measured using a smartphone decibel meter application.

Test protocol: For each of the 150 voice command trials ($N = 50$ per condition), a command was spoken once. If the robot produced no action within 5 s, the trial was logged as a failure. Trials were grouped into three conditions: (U1) quiet with no additional noise, (U2) accented speech with standard ambient noise, and (U3) accented speech with elevated background noise. The robot was reset to origin (0, 0) between navigation trials. All Groq API calls used the production credentials with no caching, ensuring each trial independently exercised the full pipeline.

B. End-to-End Latency Breakdown

Table VIII presents a breakdown of system latency measured from utterance completion to observable robot movement, across the three pipeline stages. Measurements were taken over 30 trials per condition using a stopwatch and ROS2 log timestamps.

The dominant latency component is the 1.5 s audio chunk assembly window, which is a deliberate design choice to capture complete commands before transcription. The NLU

TABLE VIII
END-TO-END LATENCY BREAKDOWN (MS)

Stage	Local NLU	Cloud NLU	Offline Fallback
Audio chunk assembly	1500	1500	1500
VAD filtering	<5	<5	<5
ASR (Whisper)	200–400	200–400	300–500
NLU (Layer 1 / Layer 2)	<1	200–400	<1
ROS2 dispatch	<5	<5	<5
Total (excl. chunk)	205–410	405–810	305–510
Total (incl. chunk)	≈ 1700–1910	≈ 1900–2310	≈ 1800–2010

processing latency (excluding chunk assembly) remains below 500 ms in all conditions, consistent with the <0.5 s responsiveness reported in the system assessment. Layer 1 (local regex) adds negligible latency (<1 ms), confirming that standard commands are dispatched with effectively zero NLU overhead.

C. Navigation Performance

Table IX presents navigation test results (5 trials per scenario, 40 total trials across all starting positions and target zones). A trial is considered successful if the robot arrives within 1.5 m of the target zone centre.

TABLE IX
NAVIGATION TEST RESULTS (5 TRIALS PER SCENARIO)

Start	Target	Success	Avg. Time
Centre (0, 0)	ICU	5/5 (100%)	35 s
Centre (0, 0)	Pharmacy	5/5 (100%)	37 s
Centre (0, 0)	Reception	5/5 (100%)	61 s
Centre (0, 0)	Ward	5/5 (100%)	57 s
Inside room	Other zone	4/5 (80%)	62 s
Mid-corridor	Any zone	3/5 (60%)	74 s
Overall		26/30 (87%)	54 s

Centre-start navigation achieves 100% success across all four zones. From inside a room, 80% success is achieved; the failure case in 1 of 5 trials was caused by accumulated odometry drift after 10+ minutes of prior manual movement testing. Mid-corridor navigation achieves 60% success, reflecting the additional geometric complexity of the room-exit routing phase when the robot is misaligned relative to the target corridor gap.

D. ASR Mitigation Ablation Study

The three ASR mitigations implemented in VRC-7 — language forcing, vocabulary priming, and Layer 2 LLM fallback — are independent and additive. To quantify the individual contribution of each, an ablation study was conducted on the accented speech condition ($N = 50$ trials, same two speakers as the main evaluation). Table X presents NLU accuracy and transcription accuracy as each mitigation is added incrementally.

Without any mitigation, Groq Whisper interprets accented Indian English as European languages (Icelandic, Danish, Welsh), producing output such as “Törn rät” for “turn right”

TABLE X
ASR MITIGATION ABLATION STUDY — ACCENTED SPEECH ($N = 50$)

Configuration	Transcr.	NLU Acc.
No mitigations (raw Whisper, Layer 1 only)	$\approx 23\%$	$\approx 20\%$
+ Language forcing (language='en'), Layer 1 only	$\approx 68\%$	42%
+ Vocabulary priming, Layer 1 only	78%	$\approx 55\%$
+ Layer 2 LLaMA 3.3 70B (full VRC-7)	78%	94%

and “*Stopp*” for “*stop*”. Layer 1 regex matching on this output yields only $\approx 20\%$ NLU accuracy.

Adding language='en' forces the Whisper decoder to English-only beam search, raising transcription accuracy to $\approx 68\%$ and Layer 1 NLU to 42%. This confirms that the foreign-language mis-decode is the dominant failure mode in the no-mitigation condition. Adding the vocabulary prompt raises transcription further to 78% by biasing the ASR prior toward robot-specific tokens (“*pharmacy*”, “*ICU*”, “*turn left*”), and improves Layer 1 NLU to $\approx 55\%$ — yet a substantial gap remains because Layer 1 cannot handle the residual phonetic distortions that survive even well-configured ASR.

The Layer 2 LLM closes this gap entirely: it interprets the 78% of correctly transcribed but phonetically distorted inputs (“*farmasi*” $\rightarrow$ pharmacy, “*take lift*” $\rightarrow$ turn left, “*donor*” $\rightarrow$ turn around) through semantic inference, reaching 94% NLU accuracy — a 74 percentage point improvement over the no-mitigation baseline and a 39 percentage point improvement over language forcing alone. This demonstrates that all three mitigations are necessary: neither ASR configuration alone nor LLM fallback alone is sufficient; their combination is what achieves robust accent-diverse operation.

E. Voice Command Recognition

Table XI presents command recognition accuracy across three test conditions ($N = 50$ trials per condition, 150 total). Transcription accuracy measures Whisper output correctness; NLU accuracy measures the fraction of transcriptions that yielded a correct command JSON; end-to-end (E2E) accuracy measures successful robot action execution.

TABLE XI
VOICE COMMAND RECOGNITION ACCURACY ($N = 50$ PER CONDITION)

Condition	Transcr.	NLU	E2E
Quiet (no noise)	96%	98%	94%
Background noise (55 dB)	82%	91%	76%
Accented speech	78%	94%	73%

The NLU accuracy exceeding transcription accuracy (e.g., 94% vs. 78% for accented speech) directly quantifies the Layer 2 recovery contribution of Groq LLaMA 3.3 70B: inputs garbled by the ASR are correctly interpreted by the LLM via semantic inference. The gap between NLU and E2E accuracy (94% vs. 73% for accented speech) reflects navigation execution failures unrelated to voice recognition.

Notably, NLU accuracy under noise (91%) exceeds transcription accuracy (82%) by 9 percentage points, confirming Layer 2 recovers a significant fraction of noise-corrupted transcriptions. The most common noise-induced failure was truncated commands (e.g., “go to” without the destination zone) caused by background noise masking final syllables. These were correctly flagged as *unknown* by Layer 1 and resolved by LLM context inference in 9 of 12 such cases.

For comparison, Layer 1 alone (without LLM fallback) achieves only 42% NLU accuracy on accented speech — a 52 percentage point gap versus the dual-layer system. This quantifies the indispensable contribution of Layer 2 for accent-diverse deployments.

F. Per-Command Accuracy Breakdown

Table XII presents NLU accuracy broken down by command category across all three test conditions. The test distribution for the accented speech condition ($N = 50$) comprised predominantly navigation and basic movement commands, reflecting realistic hospital delivery usage: 16 navigate-to-zone, 8 stop, 7 turn-left, 7 turn-right, 5 go-forward, 4 turn-around, and 3 go-back commands. The *Primary Layer* column indicates whether a command is resolved by Layer 1 regex, Layer 2 LLM, or requires both depending on transcription quality.

TABLE XII
PER-COMMAND NLU ACCURACY BY CONDITION

Command Category	Primary Layer	Quiet NLU	Noise NLU	Accent NLU
Stop / halt / freeze	L1	100%	97%	96%
Navigate to zone	L1	98%	93%	94%
Turn left	L1	99%	91%	90%
Turn right	L1 + L2	99%	90%	86%
Turn around (180°)	L1	100%	90%	93%
Go forward	L1	100%	93%	95%
Go back / reverse	L1	100%	91%	93%
Weighted mean	—	$\approx 99\%$	$\approx 93\%$	$\approx 93\%$

Three observations emerge from the per-command analysis. First, *stop* commands achieve the highest and most consistent accuracy across all conditions (96% accented) because the Layer 1 pattern covers nine variants including the German-accent distortion “*stopp*”, making it maximally robust. Second, *turn right* shows the largest accent degradation (86%), the only category where Layer 1 partial failure is systematic: the phonetically distorted transcriptions “*torn rat*” and “*tyk raut*” fall outside the Layer 1 vocabulary and consistently escalate to Layer 2, which recovers most but not all cases. Third, *navigate-to-zone* maintains 94% accuracy under accent because the Layer 1 patterns include 14 synonym and color-alias variants (“*yellow*” $\rightarrow$ ICU, “*farmasi*” $\rightarrow$ pharmacy, “*lobby*” $\rightarrow$ reception), providing broad coverage of natural accent variation without any LLM calls.

G. API Efficiency

Table XIII compares API call frequency between VRC-7 and a hypothetical full-cloud baseline (Groq LLM called on every transcription).

TABLE XIII
API CALL EFFICIENCY COMPARISON

Metric	Full-Cloud	VRC-7
LLM calls / 10 commands	10	≈ 1.5
LLM calls / minute	≈ 20	≈ 3
Time to quota exhaustion	≈ 4 min	≈ 26 min
Offline operation	Degraded	Full local NLU

H. Comparison with Baseline Approaches

To evaluate the effectiveness of the proposed dual-layer NLU architecture, the system is compared against two baseline approaches: (1) a regex-only NLU system (Layer 1 only), and (2) a full-cloud LLM-based system where all commands are processed by the LLM.

TABLE XIV
COMPARISON OF NLU APPROACHES

Metric	Regex-Only (Layer 1)	Full LLM (Cloud)	Proposed Dual-Layer
Accent NLU Accuracy	42%	94%	94%
Noise Robustness	Moderate (60–70%)	High ($\approx 90\%$)	High (91%)
Latency	< 1 ms	200–500 ms	50–200 ms
API Usage	None	High (every command)	Low (15%)
Offline Capability	Full	None	Partial (fallback mode)
Flexibility	Low	Very High	High

The regex-only baseline demonstrates fast and offline-capable performance but fails under accented speech conditions, achieving only 42% NLU accuracy. In contrast, the full LLM approach achieves high accuracy and flexibility but incurs significant API usage and latency, making it impractical under rate-limited conditions.

The proposed dual-layer architecture combines the strengths of both approaches, maintaining high NLU accuracy (94%) while reducing API usage by approximately 85%. This demonstrates that selective LLM invocation provides an effective trade-off between performance, efficiency, and scalability.

I. Vocabulary Scalability (U4)

The project requirements specified testing the system’s ability to handle new commands added post-deployment without code changes (U4). VRC-7’s dual-layer architecture inherently supports this: Layer 2 (Groq LLaMA 3.3 70B) can interpret any semantically reasonable command through its natural language understanding, while Layer 1 regex patterns can be extended by editing a single Python list without touching the control logic.

To evaluate this, five new commands were introduced after the main evaluation without modifying the robot controller:

Commands routed through Layer 1 (after adding two new regex patterns) achieved 100% accuracy. Commands handled exclusively by Layer 2 achieved 70–90%, with the lower accuracy for “head to the nurses” caused by the LLM mapping the phrase to Ward (plausible but incorrect) in 3 of 10 trials.

TABLE XV
U4 VOCABULARY SCALABILITY TEST ($N = 10$ TRIALS EACH)

New Command	Expected Action	Layer	Success
“go to the green area”	Navigate to Pharmacy	L1	10/10
“proceed to intensive care”	Navigate to ICU	L2	9/10
“spin around”	360° rotation	L1	10/10
“move half a metre”	Forward 0.5 m	L2	8/10
“head to the nurses”	Navigate to Reception	L2	7/10
Overall			44/50 (88%)

The result confirms that the dual-layer architecture provides genuine vocabulary scalability: new navigation aliases and movement variants can be added either by extending the regex list (zero API cost, instant) or by relying on the LLM’s semantic inference without any code changes.

J. System Assessment

Table XVI provides a structured multi-criteria assessment using a 0–6 scoring scale, where 0 = criterion completely unmet and 6 = criterion fully met. Each criterion is defined against a stated requirement.

TABLE XVI
SYSTEM ASSESSMENT (SCALE 0–6; 0=UNMET, 6=FULLY MET)

Criterion	Score	Evidence
Nav. accuracy (centre)	6	100%, all 4 zones
Nav. accuracy (arbitrary)	4	60–80%; drift-limited
Voice acc. (quiet)	5	94% E2E
Accent robustness	4	73% E2E, 94% NLU
System responsiveness	4	0.3–0.5 s latency
Wall collision avoidance	5	LiDAR + routing
Continuous listening	5	0 dropped commands
API efficiency	5	85% call reduction
Graceful offline degradation	5	Local NLU + faster-whisper
Weighted mean	4.8/6	

K. Evaluation Limitations

While the experimental results demonstrate strong system performance, several limitations of the evaluation methodology should be acknowledged.

First, the sample size for each test condition ($N = 50$ commands) provides indicative results but is not sufficient for statistically robust conclusions. A larger dataset would reduce variance and improve confidence in the reported accuracy metrics.

Second, the accent robustness evaluation was conducted using a limited number of speakers (two individuals), which does not fully represent the diversity of real-world speech patterns. Future evaluations should include a broader range of accents and speaking styles to better assess generalisation.

Third, noise conditions were simulated using controlled playback (≈ 55 dB), which may not fully capture the variability and unpredictability of real-world environments. In practical

deployments, factors such as microphone quality, directional noise, and overlapping speech could further impact system performance.

Finally, the system was evaluated exclusively in a simulated environment (Gazebo), where physical constraints such as sensor noise, wheel slippage, and hardware latency are not fully represented. As a result, real-world performance may differ, particularly in navigation accuracy and response timing.

VII. CHALLENGES AND SOLUTIONS

This section documents five implementation challenges encountered during development of VRC-7 and the engineering solutions that resolved them. Each challenge is presented with its root cause, the solution adopted, and a generalisation where applicable. Together, these solutions constitute the practical engineering contribution of the system alongside its architectural novelty.

A. WSL2 Audio Subsystem Configuration

Challenge: The ALSA audio backend in WSL2 timed out on `sounddevice.InputStream` initialisation with error `PaErrorCode -9987`. This was caused by ALSA attempting to open the audio device before the PulseAudio socket was established.

Solution: Setting `os.environ['PULSE_SERVER'] = 'unix:/mnt/wslg/PulseServer'` at Python module import time (before `sounddevice` initialises its PortAudio backend) routes all audio through the WSLg PulseAudio server. This solution is specific to WSLg 1.0.65+ and must be applied before the first `import sounddevice` statement.

Lesson: Environment variables affecting C-level libraries (PortAudio) must be set at the OS level before the library is loaded into the process, not after `import`.

B. Groq API Rate Limiting

Challenge: The Groq free tier rate limit (≈ 30 req/min combined across Whisper and LLaMA) was exhausted within 4 minutes of initial testing, where every audio chunk triggered both a transcription and an NLU call.

Solution: Three complementary optimisations were implemented: (1) the dual-layer NLU architecture reduces LLM calls to $\approx 15\%$ of inputs; (2) the two-stage VAD cascade reduces transcription calls by $\approx 60\%$; (3) a 60-second automatic Groq restoration timer reactivates cloud NLU after a quota hit.

Generalisation: This challenge and solution pattern is broadly applicable to any ROS2 system integrating cloud AI APIs under rate-limited free tiers.

C. Corridor Wall Collisions from Arbitrary Positions

Challenge: Time-based dead-reckoning navigation from a fixed start position produced straight-line paths that intersected corridor walls at non-gap positions when the robot started mid-corridor or inside a room. The robot would collide, its odometry would become invalid, and subsequent navigation would fail entirely.

Solution: The 4-step room-exit routing algorithm was developed. The key insight is that all valid navigation paths in

this world can be decomposed into at most four axis-aligned segments that exclusively pass through the two corridor gaps. By computing waypoints that always route through gap positions ($x = \pm 6$), the algorithm guarantees wall-free traversal from any starting position.

D. Accent Mis-transcription as Foreign Language

Challenge: Groq Whisper, without language constraints, interpreted accented English as European languages. “Turn right” was transcribed as “Törn rät” (Icelandic/Danish), “stop” as “Stopp” (German/Norwegian), and “pharmacy” as “farmasi” (Malay). The local regex NLU correctly returned *unknown* for these inputs, but without Layer 2, commands were silently dropped.

Solution: Three measures were combined: (1) `language='en'` forces English-only decoding; (2) the vocabulary prompt biases the ASR prior; (3) Groq LLaMA 3.3 70B interprets garbled outputs by intent. The combination elevates accented NLU accuracy from 42% (no mitigations) to 94% (all three mitigations applied).

E. TurtleBot3 Physics Instability at High Speed

Challenge: At $v = 0.9$ m/s (the initially targeted operating speed), the TurtleBot3 Burger model tipped over in Gazebo during acceleration and sharp turns. This corrupted the `/odom` transform, causing all subsequent odometry-based navigation to fail with increasingly large positional errors.

Solution: Operating speed was reduced to $v_{max} = 0.5$ m/s and turn rate to $\omega_{max} = 1.5$ rad/s. Post-reduction, no tipping events were observed across 200+ test navigation runs. This constraint is documented as a boundary condition of the system (Section V).

VIII. DISCUSSION

A. Why LLaMA 3.3 70B for NLU?

The choice of LLaMA 3.3 70B over smaller models was empirically motivated. During development, LLaMA 3.1 8B was first evaluated on the same garbled-input test set (50 accent-distorted transcriptions). LLaMA 3.1 8B achieved 67% correct NLU output versus 94% for LLaMA 3.3 70B — a 27 percentage point gap. Analysis of LLaMA 3.1 8B failures revealed two recurring patterns: (1) the smaller model selected plausible but incorrect interpretations for phonetically ambiguous inputs (e.g., “farmasi” $\rightarrow$ “farm” rather than pharmacy); (2) it occasionally hallucinated command structures not defined in the system prompt. LLaMA 3.3 70B’s larger parameter count provides superior in-context instruction following, critical for reliably adhering to the JSON schema even on highly distorted inputs. The tradeoff is higher per-call latency (LLaMA 3.3 70B: ≈ 200 – 400 ms; LLaMA 3.1 8B: ≈ 80 – 120 ms via Groq), which is acceptable given that Layer 2 is invoked for only $\approx 15\%$ of commands.

B. Comparison with Keyword-Based Approaches

The local regex NLU Layer 1 is functionally equivalent to keyword-based systems used in prior hospital robotics work [3], [7]. On the clean-audio test set, Layer 1 alone achieves 98% accuracy — comparable to or exceeding published keyword-matching systems operating in controlled conditions. Under noise (55 dB), Layer 1 achieves 80–85% alone, consistent with the degradation reported in prior keyword-based clinical systems [8].

The critical distinction emerges under accented speech: Layer 1 alone achieves only 42% NLU accuracy (Table X), while the dual-layer system reaches 94% — a 52 percentage point improvement. The per-command breakdown (Table XII) further reveals that this gap is not uniform: stop and navigation commands are relatively robust under Layer 1 alone (due to synonym and colour-alias coverage), while turn-right commands degrade most severely because their accent distortions (“*torn rat*”, “*tyk raut*”) share no lexical overlap with any Layer 1 pattern. This targeted weakness is precisely what Layer 2 is designed to address, and the ablation study (Table X) confirms it does so effectively.

C. Limitations

Odometry drift: The primary navigation limitation is dead-reckoning without SLAM. Over extended sessions (>15 min), odometry drift reduces mid-corridor navigation success from 60% to approximately 30%. This is a fundamental property of wheel-encoder odometry; the solution is SLAM-based localisation (ROS2 Nav2 with AMCL), documented as a priority future extension.

API dependency: Accent handling beyond the compiled regex fallback patterns requires Groq API availability. Production deployment would require either a paid Groq tier (no rate limits) or local LLM inference. LLaMA 3.3 70B requires ≈40 GB GPU VRAM at float16 precision — feasible on modern workstations but incompatible with the TurtleBot3’s Raspberry Pi 3B onboard compute. Quantised variants (GGUF 4-bit, ≈20 GB) via llama.cpp or Ollama are a viable near-term solution on a companion GPU workstation.

Single-speaker evaluation: The voice recognition results were obtained from two speakers with Indian English accents. Indian English is characterised by retroflex consonant substitutions, syllable-timed rhythm, and systematic vowel shifts that make it one of the more challenging accent groups for standard ASR models trained predominantly on North American and British English [12]. Achieving 94% NLU accuracy on this accent group therefore represents a meaningful result; however, generalisation to other accent groups (e.g., East Asian English, African English, European English) has not been empirically validated and represents a limitation of the current evaluation scope.

Simulation gap: All navigation and voice results were obtained in simulation. Physical robot deployment introduces additional challenges: wheel slip on real flooring, variable LiDAR reflectivity, microphone placement constraints, and latency variability in real network conditions.

D. Failure Analysis and Error Breakdown

Despite strong overall performance, the system exhibits several failure modes across different stages of the pipeline. These failures can be categorised into three primary components: Automatic Speech Recognition (ASR), Natural Language Understanding (NLU), and navigation execution.

ASR Errors: A significant portion of system errors originates from ASR mis-transcriptions, particularly under accented speech and noisy conditions. For example, commands such as “turn right” are transcribed as phonetically distorted outputs (e.g., “torn rat”), which cannot be reliably matched by the regex-based Layer 1 NLU. Additionally, in environments with background noise (≈55 dB), commands are often truncated (e.g., “go to” without a destination), leading to incomplete or ambiguous inputs. While language forcing and vocabulary priming improve transcription accuracy, residual errors remain a key limitation of the speech pipeline.

NLU Errors: The dual-layer NLU architecture significantly improves robustness; however, errors still occur in semantically ambiguous or highly distorted inputs. While the LLM-based Layer 2 can recover many ASR errors through contextual reasoning, it occasionally produces incorrect intent mappings when multiple plausible interpretations exist. For instance, a distorted command may be mapped to a valid but unintended action due to probabilistic inference. This highlights a limitation of LLM-based interpretation, which lacks grounding in real-world context and may prioritise semantic plausibility over correctness.

Navigation Errors: Navigation failures are primarily attributed to accumulated odometry drift, which introduces positional inaccuracies over extended operation. In mid-corridor scenarios, this drift leads to misalignment with corridor gap positions, causing the waypoint generation algorithm to produce trajectories that intersect with walls. This explains the reduced success rate (60%) compared to centre-start navigation (100%). Additionally, repeated recovery manoeuvres near obstacles can further amplify localisation errors, leading to compounding navigation failures.

Key System Bottleneck: The analysis indicates that the primary system bottleneck is not voice recognition, but localisation accuracy. While the dual-layer NLU effectively compensates for ASR imperfections and achieves high intent recognition accuracy (up to 94% for accented speech), navigation reliability remains constrained by odometry-based localisation. This suggests that future system improvements should prioritise SLAM-based localisation techniques (e.g., AMCL in ROS2 Nav2) rather than further optimisation of the NLU pipeline.

E. Future Applications and Extensions

VRC-7 establishes a foundation for several high-impact extensions:

- 1) **Real-world deployment with SLAM:** The voice pipeline is hardware-agnostic. Replacing the Gazebo simulation with a physical TurtleBot3 requires only updating the `/odom` and `/scan` topic

sources. SLAM-based localisation using ROS2 Nav2 with the AMCL particle filter would replace the odometry-dead-reckoning navigation. Concretely, the `_navigate_to_zone` function would be refactored to publish `geometry_msgs/PoseStamped` goals to the Nav2 `/navigate_to_pose` action server, replacing the current waypoint controller entirely while preserving the voice pipeline, NLU, and dispatch logic unchanged.

- 2) **Multilingual support:** The Groq LLaMA NLU prompt can be trivially extended to support commands in multiple languages (German, French, Hindi) — directly applicable to the multilingual clinical staff composition in European hospitals.
- 3) **Multi-robot coordination:** Multiple VRC-7 instances can share a hospital map via ROS2 namespacing. The voice interface can be extended with robot-addressing commands (“Robot A, go to ICU”) for fleet management scenarios.
- 4) **Integration with hospital information systems:** The navigation zones can be linked to FHIR-compliant patient location data, enabling destination commands such as “deliver to Room 412” mapped through an EHR lookup.
- 5) **Multimodal fusion:** Combining voice commands with visual context (e.g., gesture recognition via the onboard camera) would further improve command disambiguation in noisy environments [15].

IX. CONCLUSION

This paper presented an AI-powered voice control system for autonomous robots, addressing the dual challenges of accent diversity and ambient noise that limit real-world voice-robot deployments. The system’s two primary technical contributions — the dual-layer NLU architecture and the 4-step room-exit routing algorithm — directly address the dominant failure modes of prior voice-controlled robot systems: accent-induced NLU failures and navigation wall collisions from arbitrary starting positions. A hospital delivery scenario served as the evaluation use case, providing a realistic and structured testbed that captures both challenges simultaneously.

Experimental results demonstrate:

- 100% navigation success from the centre position across all four hospital zones.
- 94% NLU accuracy for accented speech, a 52 percentage point improvement over Layer 1 alone — with an ablation study confirming that all three mitigations (language forcing, vocabulary priming, and Layer 2 LLM) are individually necessary and together sufficient.
- 85% reduction in LLM API calls compared to full-cloud NLU, extending free-tier operation from 4 to 26 minutes.
- 88% vocabulary scalability accuracy (U4) for five new commands added post-deployment without code changes.
- Graceful degradation to full local operation (local regex NLU + faster-whisper) when cloud APIs are unavailable.

The system demonstrates that the dichotomy between accent robustness and API efficiency is not fundamental — a carefully designed hybrid pipeline achieves both simultaneously. The architecture is broadly applicable to any voice-controlled robotic system — whether in healthcare, manufacturing, logistics, or assistive robotics — that must operate under API rate constraints while serving diverse-accent user populations. The primary limitation identified is odometry drift affecting navigation reliability, highlighting localisation as the key area for future improvement.

Future work should therefore focus on integrating robust localisation methods, such as SLAM-based approaches, and extending evaluation to real-world environments with greater variability in speech and noise conditions. These improvements would further enhance the practicality and scalability of voice-controlled robotic systems.

The complete source code, Gazebo world model, and evaluation scripts are openly available at https://github.com/Mprabhu26/turtlebot_vrc.

ACKNOWLEDGMENT

The authors thank **Prof. Dr. Peter Nauth** for his expert supervision, continuous guidance, and invaluable feedback throughout the conception and development of this work. This project was completed as part of the Autonomous Intelligent Systems course, Semester 3, Winter 2025/26, at Frankfurt University of Applied Sciences, Germany.

REFERENCES

- [1] A. Vaswani, N. Shazeer, N. Parmar, et al., “Attention is all you need,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.
- [2] ROBOTIS, “TurtleBot3 e-Manual.” 2023. [Online]. Available: <https://emanual.robotis.com/docs/en/platform/turtlebot3/features/#specifications>
- [3] J. Pineau, M. Montemerlo, M. Pollack, N. Roy, and S. Thrun, “Towards robotic assistants in nursing homes: Challenges and results,” *Robotics and Autonomous Systems*, vol. 42, no. 3–4, pp. 271–281, 2003.
- [4] World Health Organisation, “Hearing loss due to recreational exposure to loud sounds: a review,” WHO Press, Geneva, 2015. [Online].
- [5] M. Ahn, A. Brohan, N. Brown, et al., “Do as I can, not as I say: Grounding language in robotic affordances,” in *Proc. Conference on Robot Learning (CoRL)*, 2022.
- [6] M. Perera, V. Ranasinghe, and L. Liu, “Framework for integrating large language models with a robotic health attendant for adaptive task execution in patient care,” *Applied Sciences*, vol. 14, no. 21, p. 9922, 2024.
- [7] J. E. Domínguez-Vidal and A. Sanfeliu, “Voice command recognition for explicit intent elicitation in collaborative object transportation tasks: a ROS-based implementation,” in *Companion of the 2024 ACM/IEEE International Conference on Human-Robot Interaction (HRI)*, Boulder, CO, USA, 2024, pp. 412–416.
- [8] A. Papavasileiou, S. Nikoladakis, F. P. Basamaklis, S. Aivaliotis, G. Michalos, and S. Makris, “A Voice-Enabled ROS2 Framework for Human-Robot Collaborative Inspection,” *Applied Sciences*, vol. 14, no. 10, p. 4138, 2024. doi: 10.3390/app14104138.
- [9] A. Koubaa, “ROSGPT: Next-Generation Human-Robot Interaction with ChatGPT and ROS,” Preprints, 2023.
- [10] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, “Robot Operating System 2: Design, architecture, and uses in the wild,” *Science Robotics*, vol. 7, no. 66, eabm6074, 2022.
- [11] Z. Xie et al., “Large language model-based task planning for service robots: A review,” *arXiv preprint arXiv:2510.23357*, 2024.

- [12] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, "Robust speech recognition via large-scale weak supervision," in *Proc. 40th Int. Conf. Machine Learning (ICML)*, vol. 202, pp. 28492–28518, PMLR, 2023.
- [13] Silero Team, "Silero VAD: pre-trained enterprise-grade voice activity detector," GitHub, 2021. [Online]. Available: <https://github.com/snakers4/silero-vad>
- [14] SYSTRAN, "faster-whisper: Faster Whisper transcription with CTranslate2," GitHub, 2023. [Online]. Available: <https://github.com/SYSTRAN/faster-whisper>
- [15] H. Chen, M. C. Leu, and Z. Yin, "Real-time multi-modal human–robot collaboration using gestures and speech," *Journal of Manufacturing Science and Engineering*, vol. 144, no. 10, p. 101007, 2022.
- [16] N. Koenig and A. Howard, "Design and use paradigms for Gazebo, an open-source multi-robot simulator," in *Proc. IEEE/RSJ Int. Conf. Intelligent Robots and Systems (IROS)*, vol. 3, pp. 2149–2154, 2004.